8g. Type Stability with Higher-Order Functions

Martin Alfaro
PhD in Economics

INTRODUCTION

Functions in Julia are **first-class objects**, a concept also referred to as **first-class citizens**. This means that functions can be handled just like any other variable: we can define vectors of functions, have functions whose outputs are other functions, and compose them in flexible ways that would be impossible if functions were treated as different entities.

In particular, the property makes it possible to define **higher-order functions**, which are functions that take another function as an argument. We've already worked with several of them, often in the form of anonymous functions passed as function arguments. A familiar example is `map(<function>, <collection>)`, which applies `<function>` to every element of `<collection>`.

Our goal in this section is to understand when higher-order functions are type-stable. As we'll discover, these functions present some challenges in this regard.

> **Remark**
> Throughout the explanations, we'll often refer to the function passed as an argument as the *callback function*.

THE ISSUE

In Julia, *each function has its own unique concrete type*. In turn, each of these concrete types is a subtype of an abstract type called `Function`. The type `Function` serves as an umbrella for all possible functions defined in Julia.

This design of types for functoins creates challenges when the compiler attempts to specialize the computation method of higher-order functions. In particular, if Julia were to generate a distinct specialized method for every possible callback function, the number of methods could grow explosively.

To address this issue, Julia adopts a conservative strategy, **often choosing not to specialize the methods of higher-order functions**. In particular, we'll see that Julia avoids specialization if the callback function isn't explicitly called. When specialization is skipped, the performance can drop sharply, sometimes resembling the overhead of executing code in global scope.

It's therefore important to recognize when specialization is inhibited and to monitor its consequences. If you notice that performance is severely impaired, there are still ways to enforce specialization. In this section, we'll explore these strategies.

AN EXAMPLE OF NO SPECIALIZATION

To understand when higher-order functions fail to specialize, let's look at a simple example: summing the transformed elements of a vector `x`. The only constraint we impose is that the transformation itself must remain generic, allowing us to apply different functions for the transformation.

To express this, we define a higher-order function `foo`, which broadcasts over `x` to transform its elements through some function `f`. To demonstrate how `foo` works, we call it with the function `abs` as the transforming function, which provides absolute values.

NO SPECIALIZATION

```
x          = rand(100)

foo(f, x) = f.(x)
```

```
julia> @code_warntype foo(abs,x)
```

Even when `foo(abs,x)` isn't specialized, `@code_warntype` **fails to detect any type-stability issues**. This happens because `@code_warntype` evaluates type stability *under the assumption that specialization is attempted*. In our example, this assumption simply doesn't hold and therefore `@code_warntype` is of no use.

The source of type instability in this case is that Julia **avoids specialization if a callback function isn't explicitly called inside the function**. In the example, the function `f` only enters `foo` as an argument of broadcasting, but there's no explicit line calling `f`.

To gather indirect evidence about the lack of specialization, we can compare the performance of the original `foo` function with a modified version that explicitly calls `f`.

NO SPECIALIZATION

```julia
x = rand(100)


function foo(f, x)
    f.(x)
end
```

```julia
julia> foo(abs, x)
100-element Vector{Float64}:
 0.9063
 0.443494
 ⋮
 0.121148
 0.20453
julia> @btime foo(abs, $x)
  847.536 ns (12 allocations: 1.250 KiB)
```

SPECIALIZATION

```julia
x = rand(100)

function foo(f, x)
    f(1)                # irrelevant computation to force specialization
    f.(x)
end
```

```julia
julia> foo(abs, x)
100-element Vector{Float64}:
 0.9063
 0.443494
 ⋮
 0.121148
 0.20453
julia> @btime foo(abs, $x)
  28.991 ns (2 allocations: 928 bytes)
```

The comparison reveals a significant decrease in execution time when `f(1)` is added, as well as a notable reduction in memory allocations. Excessive allocations are often indicative of type instability.

FORCING SPECIALIZATION

> ⚠️ **Warning!**
>
> Exercise caution when forcing specialization. Overly aggressive specialization can degrade performance severely, explaining why Julia's default approach is deliberately conservative. In particular, you should avoid specialization when your script repeatedly calls a higher-order function with many unique functions. [1]

Explicitly calling the callback function to force specialization isn't ideal, as it introduces an unnecessary computation. Fortunately, alternative solutions exist. One of them is to type-annotate `f`, which provides Julia with a hint to specialize code for that function.

Another solution involves wrapping the function in a tuple, before passing it as an argument. This ensures the identification of the function's type, as tuples define a concrete type for each of their elements.

Below, we outline both approaches.

NO SPECIALIZATION

```julia
x     = rand(100)


function foo(f, x)
    f.(x)
end
```

```julia
julia> foo(abs, x)
100-element Vector{Float64}:
 0.9063
 0.443494
 ⋮
 0.121148
 0.20453
julia> @btime foo(abs, $x)
  840.536 ns (12 allocations: 1.250 KiB)
```

TYPE-ANNOTATED FUNCTION

```julia
x     = rand(100)


function foo(f::F, x) where F
    f.(x)
end
```

```julia
julia> foo(abs, x)
100-element Vector{Float64}:
 0.9063
 0.443494
 ⋮
 0.121148
 0.20453
julia> @btime foo(abs, $x)
  44.730 ns (2 allocations: 928 bytes)
```

FUNCTION IN TUPLE

```julia
x     = rand(100)
f_tup = (abs,)

function foo(f_tup, x)
    f_tup[1].(x)
end
```

```julia
julia> foo(f_tup, x)
100-element Vector{Float64}:
 0.9063
 0.443494
 ⋮
 0.121148
 0.20453

julia> @btime foo($f_tup, $x)
  47.829 ns (2 allocations: 928 bytes)
```

FOOTNOTES

[1.] For discussions about the issue of excessive specialization, see here and here.